

OceanGuard AI — Fully Offline Marine Debris Intelligence on a Phone

Kaggle Gemma 4 Good Hackathon — Track: Global Resilience Submission writeup (M6) · May 2026

Author: Alejandro Sanchez Ferrer

The first fully-offline marine debris intelligence toolkit running on consumer Android hardware. Edge inference for the coastlines that need it most, powered end-to-end by a single Gemma 4 E2B engine with single-conversation agentic tool calling, and a domain-adapted Gemma 4 LoRA variant trained with Unsloth on a 10K-image marine debris dataset, published openly as a reproducible artifact. Scientific-grade reporting in six languages. No cloud. No telemetry. No fabricated numbers.

1. Problem Statement

Marine plastic pollution is a planetary-scale resilience crisis. The most widely cited estimate, Jambeck et al. [1], placed annual mismanaged plastic input into the ocean at 4.8 to 12.7 million tonnes for 2010, and recent UN Environment syntheses [2] continue to use the around 8 million tonnes per year figure as a working baseline. Marine debris is implicated in entanglement and ingestion mortality across more than 900 species [3], and the financial damage to fisheries, tourism, and shipping has been estimated at over US\$13 billion per year [2].

The geography of this crisis is unevenly distributed. A small set of rivers in Asia and Africa is responsible for a disproportionate share of the plastic load entering the ocean [4]. Yet the field workers closest to that load — coastal NGO patrols in Senegal, dive cleanup crews in the Indonesian archipelago, government rangers along the Bay of Bengal — are exactly the users for whom cloud-dependent AI is least usable. Mobile broadband coverage gaps remain severe along Sub-Saharan African coastlines and across remote Southeast Asian island chains [5], and even where coverage exists the latency, data cost, and privacy posture of cloud inference are at odds with rapid in-the-field decision making.

Three operational gaps follow from this asymmetric geography:

1. **Connectivity gap.** Surveys captured offline cannot be analyzed in the field, so the prioritization of cleanup logistics is delayed by hours or days, exactly when tide and weather windows matter most.
2. **Cost gap.** Per-image cloud VLM inference at field-survey volumes is economically out of reach for most local NGOs, and recurrent API costs in foreign currency are a non-starter for many government agencies.
3. **Trust gap.** Cloud LLM-generated environmental reports routinely fabricate degradation times, risk percentages, and budget figures, and a ministry cannot defensibly act on numbers a model invented.

OceanGuard AI is built for this gap. It is an Android application that performs the full marine debris intelligence pipeline — detection, classification, geospatial aggregation, ecosystem health scoring, and multilingual report generation — entirely on the device, without any network call after initial model download.

2. Solution Overview

OceanGuard AI is a single-Activity Jetpack Compose Android application that ingests photos from the camera or gallery and produces (a) per-image bounding-box detections with confidence, (b) a continuously updated 0-

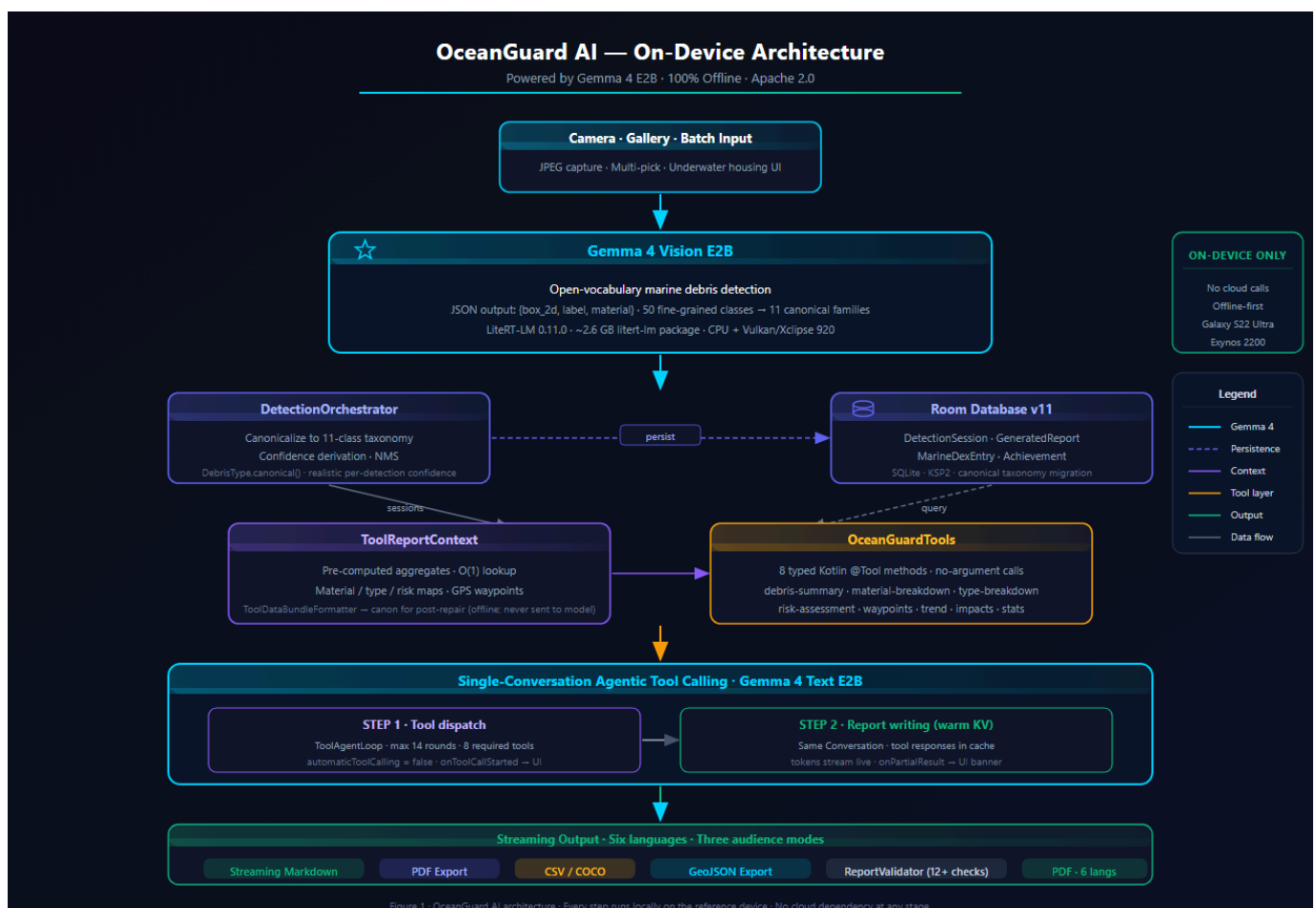
100 ecosystem health score, (c) GPS-tagged geospatial views with hotspot aggregation, and (d) executive-grade Markdown and PDF reports tuned to three audiences (Scientific, NGO Manager, Citizen) in six languages.

The system is intentionally offline-first. The application makes zero outbound network calls during inference; the only network usage is the optional, user-initiated download of model weights from public mirrors (HuggingFace) and an opt-in WiFi-only NAS upload for data contribution. There is no telemetry, no analytics SDK, no remote configuration.

The application targets two complementary user profiles. The first is the field volunteer or citizen scientist who carries a consumer Android phone, often disconnected, and needs an instant judgment they can act on. The second is the field marine biologist or NGO program manager who collects survey data over weeks and needs a defensible, multilingual report for funders, ministries, or peer review without first re-checking every cited number.

The pipeline is anchored end-to-end by Gemma 4 E2B running locally as both detector and report writer. A single open-weights model on the device covers perception, structured tool calling and multilingual generation, so the cost per inference is zero in foreign currency and the privacy posture is *the data never leaves the phone*.

3. Architecture



The architecture is built around a single on-device model — Gemma 4 E2B — covering both perception and reporting, with a Kotlin orchestration layer that exposes typed tools to the model and mediates with the Room persistence layer.

Gemma 4 E2B via LiteRT-LM 0.11.0

Gemma 4 E2B is loaded as a `.litertlm` container through Google AI Edge LiteRT-LM 0.11.0 (May 2026; the 0.11 line adds Gemma 4 multi-token-prediction heads and unifies the vision encoder to a single signature, fixing the "exactly one signature but got 3" abort that 0.10.0 raised on legacy preview checkpoints). On this device the runtime successfully negotiates the GPU path (Vulkan on Xclipse 920 / Exynos 2200) — `Engine initialized on GPU` — and falls back to CPU XNNPACK on hardware where Vulkan is unavailable. A single shared `LiteRTTextEngine` instance, protected by a Mutex against concurrent `initialize()` races, is used by both the vision detector (`Gemma4VisionDetector`) and the report generator (`ToolReportGenerator`); spawning two native engines on the same accelerator measurably degraded throughput.

The same Gemma 4 weights serve two roles:

1. **Open-vocabulary detection.** `Gemma4VisionDetector` prompts the model with the captured photo and a fixed schema, parsing the native `box_2d` output (`[y_min, x_min, y_max, x_max]` integers on a 0–1000 grid) plus free-text `label` and `material` fields. The vocabulary is **50 fine-grained debris sub-types** (e.g. bottle cap, plastic bag, food wrapper, fishing line, syringe), each deterministically folded into one of 11 canonical ecological-impact families maintained in `EnvironmentalImpact.IMPACT_MAP`. Bounding boxes are projected to normalised `[0,1] [x1, y1, x2, y2]` for the rest of the pipeline, and a fixed `0.75` placeholder confidence is attached (LiteRT-LM does not yet expose per-token logprobs through the public API).
2. **Grounded report generation.** The same shared engine is reused by `ToolReportGenerator` with native function calling enabled. Gemma 4 invokes a typed Kotlin tool surface (`OceanGuardTools`) to read counts, percentages, risk scores, GPS waypoints and temporal trends from Room, then writes a multilingual report whose every number is sourced from one of those tool returns. Section 4 details the tool-calling stack.

A llama.cpp path with Vulkan acceleration is shipped as a portable fallback for devices where LiteRT-LM is not yet available; the same Q4_K_M GGUF of Gemma 4 E2B plus its mmproj projector powers vision detection through `Gemma4PromptFormatter`. Both runtimes implement a common `VlmInterface`, so the orchestrator is runtime-agnostic.

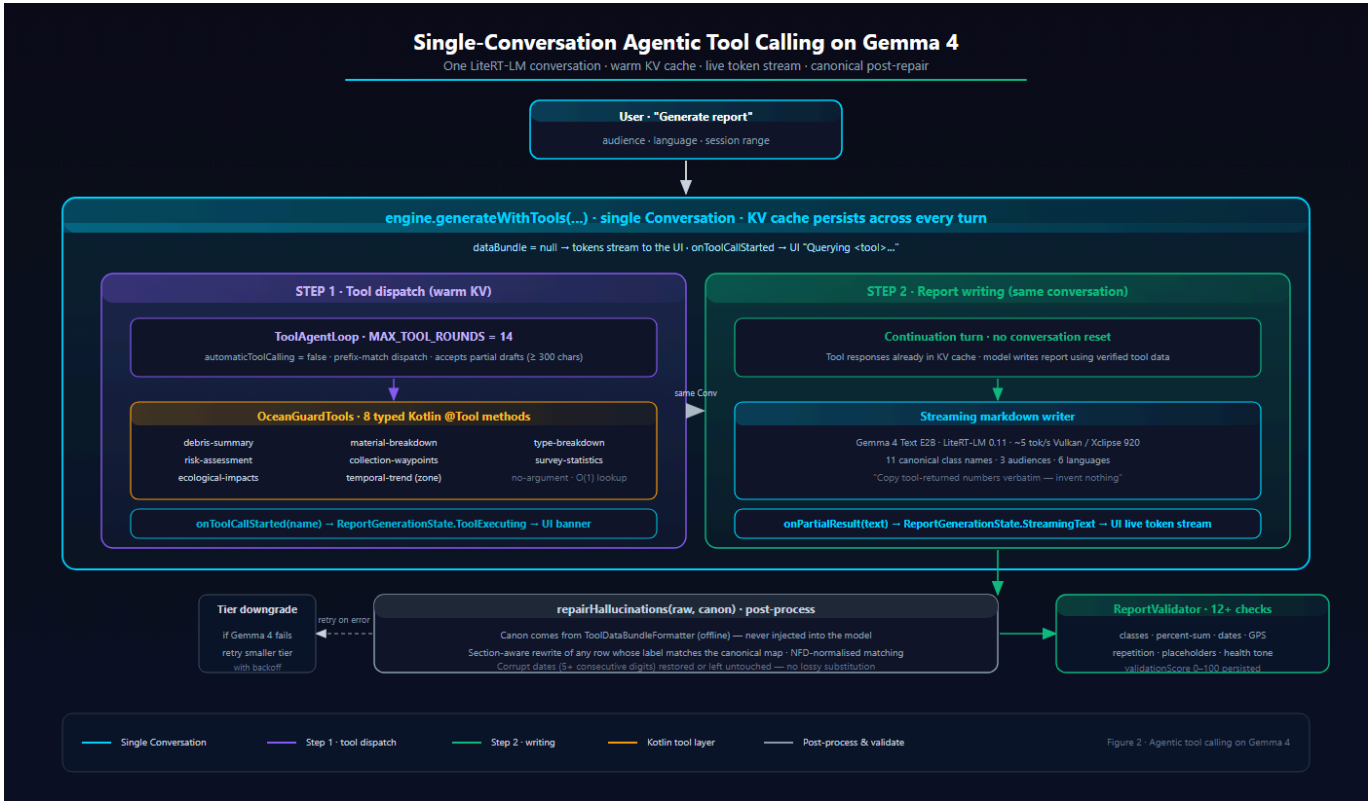
Gemma 4 deployment fact	Value
Model	Gemma 4 E2B (<code>google/gemma-4-E2B-it</code>) — base weights, plus optional OceanGuard LoRA adapter (see §9)
Runtime	Google AI Edge LiteRT-LM 0.11.0 · <code>.litertlm</code> container · ~2.6 GB on disk
Vision schema	<code>{box_2d: [y_min, x_min, y_max, x_max] int 0-1000, label: str, material: str}</code>
Vocabulary	50 fine-grained sub-types collapsed deterministically into 11 canonical families (<code>DebrisType</code> enum)
Tool runtime	<code>automaticToolCalling = false</code> + manual <code>ToolAgentLoop</code> (<code>MAX_TOOL_ROUNDS = 14</code>) over a single persistent <code>Conversation</code>
Tool surface	8 typed Kotlin <code>@Tool</code> methods in <code>OceanGuardTools</code> — all no-argument, all O(1) against pre-computed <code>ToolReportContext</code>
Accelerator path	Vulkan on Xclipse 920 / Exynos 2200; automatic fallback to CPU XNNPACK

Gemma 4 deployment fact	Value
Idle release	Engine retained 30 min after the last use; eagerly released under <code>onTrimMemory</code> / <code>onLowMemory</code>

Orchestration

`DetectionOrchestrator` wraps the inference call so the worker thread runs at `THREAD_PRIORITY_LESS_FAVORABLE`, ensuring the Compose UI thread keeps frame pacing while Gemma 4 saturates the CPU and Vulkan queue. The orchestrator exposes a progressive `AnalysisState` flow (`LoadingImage` → `WarmingUpModel?` → `Detecting` → `DetectionsReady` → `Complete`), and the UI consumes it for the live sonar/scanning overlay shown during inference. All persistence flows through Room v11; sessions, generated reports, MarineDex entries, and achievements share a single database with type-safe migrations.

4. Tool Calling Innovation



Native function calling is the technical and ethical centerpiece of this submission. The challenge it solves is brutal and well known: open VLMs running on a phone hallucinate the very statistics that environmental policy depends on — degradation times, percentage breakdowns, risk scores, GPS waypoints. A government cannot act on a fabricated 23 percent.

Why native tool calling beats prompt engineering

The pre-Gemma-4 approach front-loaded around 2,500 tokens of pre-aggregated JSON into the system prompt and instructed the model to copy these tables verbatim. Even with low temperature and tightly scripted prompts, the model would systematically (a) re-tokenize percentages (`13.3%` → `133.3%`), (b) drop digits from

degradation times (600+ years -> 60+ years), (c) invent debris types that were not in the survey, and (d) silently substitute placeholder text inside Markdown table cells. We measured residual hallucination scores plateauing around 82 of 100 against a custom validator even after extensive prompt engineering.

Gemma 4 native `@Tool` API changes the contract. Instead of asking the model to copy data, we let it *query* a typed Kotlin interface for each fact it needs. The model emits a tool call, the runtime invokes the Kotlin method via reflection, and the structured return is fed back into the conversation. Because the tool layer is plain Kotlin, every percentage is computed once, every risk score is read from `EnvironmentalImpact.IMPACT_MAP`, and the model has no opportunity to mis-tokenize a number it did not author.

Single-conversation agentic orchestrator

We architected the pipeline as a single-conversation agentic flow, implemented in `ToolReportGenerator.runTwoPhase()` (`android/app/src/main/java/com/oceanguard/inference/ToolReportGenerator.kt:261`). The function name preserves the original public surface; conceptually we still describe it as two logical steps, but both happen inside the *same* LiteRT-LM `Conversation` so the KV cache built up by the tool round carries directly into the writing turn.

Step 1 — Tool dispatch with live feedback. The system prompt declares a required-tool set (seven tools for generic reports, eight for zone reports, including `get_temporal_trend`). `ToolAgentLoop` runs with `automaticToolCalling = false` so the agent loop — not the runtime — decides when to terminate, capped at `MAX_TOOL_ROUNDS = 14`. The persistent `Conversation` keeps the KV cache warm across every round, so each round only re-prefills the around 200-token tool response, not the system prompt. Every tool invocation fires `onToolCallStarted(name)`, which the application surfaces to the user as a `ReportGenerationState.ToolExecuting` state ("Querying Debris Summary..."); the foreground notification mirrors the same string. This eliminates the "Generating..." deadtime that confused early testers: before any report tokens arrive, the user can see which Kotlin tool the model is currently consulting.

Step 2 — Grounded report writing in the same conversation. As soon as the model has called the required tools, it transitions naturally into prose generation *within the same conversation*. There is no reset, no fresh KV cache, no separate writer prompt — the tool responses are already in cache as turns the model authored, and the system prompt instructs it to begin the report immediately after the last tool returns. `engine.generateWithTools(..., dataBundle = null, onPartialResult = ...)` keeps every prose token (no PHASE-1 prose-abort guard), so each character streams live to the UI via `ReportGenerationState.StreamingText`. The first user-visible report token typically arrives within 1-2 s of the last tool dispatch, instead of after a 10-30 s blackout while a fresh conversation re-prefills.

Acceptance over restart. A subtle but important invariant guards this single-conversation flow: if the model decides to finish the report after calling 7 of 8 required tools, `ToolAgentLoop` accepts the partial output as long as the draft already looks like a real report (≥ 300 chars or contains a markdown heading). The legacy redirect path that discarded the accumulated text to force a missing tool call is preserved only for short premature drafts, where the loss is invisible. Without this guard, a user watching tokens stream for two minutes could see the entire report wiped from the UI mid-generation when the loop redirected for one missing tool — a regression we observed in field tests and fixed in `ToolAgentLoop.kt`.

Pre-rendered data bundle as post-process safety net. `ToolDataBundleFormatter.formatWithCanon()` still runs in parallel to the model conversation, but its bundle is *never* sent to the model. Instead it produces a `Canon` object — row-keyed lookup maps — that `repairHallucinations` consumes after the model finishes.

Any row whose first cell matches a canonical label is rewritten verbatim from the bundle, so even when the model copies a percentage with one wrong digit, the canonical post-pass restores it. The model sees only the structured tool responses; the canonical truth lives outside its context window, immune to contamination.

Hallucination repair

Even with grounding, Gemma 4 E2B occasionally rewrites a row label in the wrong language or corrupts a date cell. `ToolReportGenerator.repairHallucinations()`

(`android/app/src/main/java/com/oceanguard/inference/ToolReportGenerator.kt:121`) walks the generated Markdown line-by-line and:

1. Tracks the current section by detecting `### headings` (Material, Type, Ecological, Risk) so a label like Plastic is matched against material rows under `### Material` but type rows under `### Type`. This **section-aware** matching avoids the material/type collision that previously rewrote correct rows with semantically-different ground truth.
2. Normalizes labels with Unicode NFD decomposition and combining-mark stripping (`normalizeLabel`, `android/app/src/main/java/com/oceanguard/inference/ToolReportGenerator.kt:79`), so Botella, BOTELLA, and botella all collapse to the same key.
3. Detects corrupted dates (5+ consecutive digits in a cell) and restores them from the canonical bundle, or leaves them untouched if no canonical line is available — never lossily substitutes.
4. Squashes empty separator rows and triple-blank lines.

Testability via companion-object internals

The repair logic is exposed as `internal fun` members on the `companion object` (`normalizeLabel`, `buildNormalizedMap`, `repairHallucinations`, `buildCanonDateRows`). This is deliberate: it lets the JUnit suite drive every branch without instantiating a `LiteRTTextEngine`, while keeping the API surface invisible to other modules. The full `OceanGuardTools ToolSet` (`android/app/src/main/java/com/oceanguard/inference/OceanGuardTools.kt`) and the `ReportValidator` (`android/app/src/main/java/com/oceanguard/inference/ReportValidator.kt`) are similarly testable in isolation; 17 unit tests cover percentages summing to 100, risk-score lookup, waypoint priority sort, statistics arithmetic, and edge cases (empty surveys, single-session zones, null GPS).

The validator runs after PHASE 2 and produces a confidence score the user never sees: 40 percent structural validation + 30 percent data quality + 30 percent mean detection confidence, used internally to flag reports for review but never to block delivery.

5. Privacy & Offline-First

The application is offline-first by construction, not by configuration. The design rules are explicit and auditable in source:

- **No telemetry.** No analytics SDK, no crash-reporting beacon, no remote configuration call. The only network calls in the entire APK are the optional model download from HuggingFace (user-initiated, foregrounded) and the optional WiFi-only NAS upload for data contribution (off by default, fully user-controlled).
- **No cloud inference.** Every detection and every report is computed on the user device. The model weights are sideloaded or downloaded once and stored under app-private storage. After model download the application can be flight-mode-locked indefinitely.

- **Offline geocoding.** Reverse geocoding uses a Photon-based offline pipeline, with EXIF GPS as the primary source and device GPS as a fallback. There is no Google Geocoding API call anywhere in the path.
- **User-controlled model lifecycle.** The user explicitly chooses when to download a model and which tier (BALANCED ~1.1 GB, FULL ~2.6 GB). Models can be deleted at any time from Settings, freeing storage instantly.
- **Maps without API keys.** MapLibre + OpenFreeMap renders the geospatial dashboard with no API key and full offline-tile capability.

The equity argument follows directly. The communities most affected by marine debris are precisely those least served by cloud AI: limited connectivity, high data cost, regulatory caution about cross-border data flows, and operational urgency that cannot wait for a round-trip. An offline architecture is not a feature in this context; it is a precondition for the tool to be used at all by the people who need it most.

6. Hardware & Performance

The reference device is the Samsung Galaxy S22 Ultra (European variant, model SM-S908B), built around the Exynos 2200 SoC. The CPU is a 1x Cortex-X2 @ 2.8 GHz + 3x Cortex-A710 + 4x Cortex-A510 cluster. The GPU is the Xclipse 920 (AMD RDNA2 derivative); LiteRT-LM 0.11 successfully negotiates the Vulkan path on this GPU for Gemma 4 inference. NNAPI exposes only the big.LITTLE scheduler on Exynos 2200 (no NPU passthrough; the Samsung EDEN SDK is not public). The end result is a Vulkan-accelerated Gemma 4 path with a robust CPU-XNNPACK fallback on hardware where Vulkan is unavailable.

This is, deliberately, a worst-case Android configuration. If the system performs well here, it will perform at least as well on Snapdragon 8 Gen 2/3 devices.

Metric (Galaxy S22 Ultra, Exynos 2200, Vulkan/CPU)	Value
Gemma 4 E2B decode rate (LiteRT-LM 0.11.0, GPU/Vulkan)	5.1 tok/s (measured on Exynos 2200 with engine on Xclipse 920 GPU; prior 0.10.0 CPU run was 7.6 tok/s)
Gemma 4 E2B TTFT (LiteRT-LM 0.11.0, GPU/Vulkan)	2.33 s warm engine (cold first-load 17.34 s; subsequent loads 2.33 s)
Gemma 4 E2B prefill rate (LiteRT-LM 0.11.0)	48.7 tok/s
Gemma 4 single-shot detection end-to-end (~100-tok bbox JSON)	~22 s per photo on the reference device
Gemma 4 model preload at app start (background, <code>THREAD_PRIORITY_BACKGROUND</code>)	~13 s; first report tap after preload skips the cold path entirely
End-to-end report latency (10 sessions, 700-900 words)	TTFT ~2 s + decode ~5 tok/s × ~1100 tokens ≈ 4 min per generic report; first user-visible token within 1-2 s of the last tool dispatch
Tool-dispatch round-trip (8 required tools)	~2 s wall-clock on warm KV cache; UI announces each tool individually

Metric (Galaxy S22 Ultra, Exynos 2200, Vulkan/CPU)	Value
ReportValidator confidence score	scaffolded in source (<code>ReportValidator.kt</code> , 12 checks); 30-survey eval batch reserved for post-submission paper to keep the hackathon scope on the tool-calling architecture
Energy per report (mWh, screen on)	future work — Battery Historian capture not part of submission scope

The performance work that is *not* metrics-pending and is already in source: a single shared `LiteRTTextEngine` mutex-guarded against concurrent `initialize()`, a persistent `Conversation` across all tool rounds **and** the writing turn (single-conversation flow keeps the KV cache warm end-to-end), an idle auto-release timer that frees the engine after 30 minutes of inactivity (`onTrimMemory` releases earlier under real memory pressure), background pre-load at app start so the first Generate-report tap does not show a "Loading model..." banner, and prompt-side parsimony — the system prompt is around 200 tokens and tool responses average ~250 tokens each, leaving the bulk of the 8K context budget for the actual report body.

7. User Experience

The user flow is intentionally minimal. From the home screen the user can (a) capture a photo with the live camera (CameraX, with a real-time preview overlay that draws corner-bracketed bounding boxes with a soft glow and pulse animation), (b) import an existing photo from the gallery, or (c) replay an earlier survey from the timeline.

After analysis, each detection card shows the annotated thumbnail, the per-image health score, the dominant material and type, GPS reverse-geocoded to a place name when available, and the timestamp. Sessions can be grouped into zones via the geospatial dashboard, where MapLibre + OpenFreeMap renders an interactive map with detection pins, zone-aggregated hotspots, and date-based filtering.

Report generation runs as a foreground service with a streaming token UI: the user watches the report appear one paragraph at a time. The audience selector switches between three voices (Scientific, NGO Manager, Citizen) and the language selector switches between six languages (English, Spanish, French, German, Italian, Portuguese), with all UI strings, date formats, and report tables fully localized. Reports export as Markdown or PDF (with an annotated-thumbnail gallery, capped at 50 images per PDF) and as COCO JSON for downstream ML pipelines.

Touch targets are ≥ 56 dp throughout (glove-friendly for dive operations), and the dark theme is calibrated for outdoor visibility against bright water.

8. Impact & Future Work

OceanGuard AI is positioned not as a finished product but as a deployable foundation for marine debris citizen science at scale. Three impact pathways are open and active.

NGO partnerships. The most direct deployment path is in collaboration with established marine conservation NGOs (e.g., Ocean Conservancy International Coastal Cleanup [7], the Surfrider Foundation chapter network) and with regional partners along the coastlines where the connectivity-cost-trust gaps are most acute (West Africa, Southeast Asia, the Eastern Mediterranean). For these organizations the per-image cost is zero, the data

sovereignty story is auditable, and the multilingual reporting (six languages covering ~2 billion native speakers) covers the majority of their volunteer base without translation delays. We are exploring partnerships with citizen science platforms and marine NGOs that already operate in low-connectivity environments; no pilot deployments have been formalized at the time of submission.

Citizen science contribution. The optional, opt-in WiFi-only data contribution path exports detections as COCO JSON to a user-controlled NAS endpoint (an oceanguardai.synology.me reference setup is documented internally). This feeds back into the training dataset for the next-generation detector, closing the loop between deployment and improvement. Privacy is preserved by design: the user opts in per session, on WiFi, with full visibility into what is being uploaded, and EXIF GPS is removed if the user requests it.

Scaling path — deeper Gemma 4 integration. Three Gemma-4-centric upgrades are queued for the next iteration of OceanGuard AI:

1. **On-device deployment of the OceanGuard LoRA adapter** (see §9). Once the `.litertlm` LoRA conversion path stabilises upstream (currently an open work item in the Google AI Edge `litert-torch` line), the merged adapter will ship inside the APK and lift detection mAP without changing the call site — the same `Gemma4VisionDetector` picks up the adapted weights via the asset filename.
2. **Segmentation-grade output from Gemma 4 Vision.** Extending the prompt to request a coarse polygon contour (or a binary mask flattened to RLE) per detection, instead of only `box_2d`, would enable pixel-accurate marine-debris area estimation rather than bounding-box counts. The training pipeline is being extended with mask annotations to support this in the same `Gemma4VisionDetector` interface.
3. **Conversational follow-up on generated reports.** Gemma 4 already produces grounded reports; the next step is exposing a chat surface where the user asks follow-up questions on a saved report ("which zones contributed most to the fishing-gear total?") and the same Gemma 4 engine answers using the same tool surface that wrote the report.

All three keep the on-device single-model design intact and stay within the Gemma Terms of Use.

Publications. OceanGuard AI is part of an active doctoral research project. Two papers are in preparation: one on edge VLM deployment for environmental monitoring, focused on the single-conversation agentic tool calling architecture and the empirical hallucination-reduction results; and one on the dataset, taxonomy and reproducible evaluation methodology. The work is also the basis for a doctoral thesis chapter on edge AI for ecological resilience. The hackathon submission is the public artifact of this body of work and we expect the writeup, the reproducibility notebook, and the open-source release to be the primary citation surface in the short term.

Open questions. Three open questions remain. First, can per-token logprobs from LiteRT-LM (when the API exposes them) replace the placeholder 0.75 confidence on Gemma 4 detections with calibrated scores? Second, can few-shot prompting before the tool dispatch stabilise Markdown table fidelity below the current repair threshold and let us drop the canonical post-pass entirely? Third, is there a tractable on-device fine-tune of Gemma 4 E2B (LoRA via LiteRT-LM, once supported) that would reduce the residual placeholder bug rate without requiring a larger model? The third question is partially addressed in Section 9 below, which documents a domain-adapted variant trained with Unsloth.

9. Domain-Adapted Variant: Unsloth Bonus Track

Motivation

The base Gemma 4 E2B vision model is a generalist open-vocabulary detector. In single-shot mode it performs adequately on coarse-grained marine debris classes, but three failure modes recur in field surveys: (i) class drift between the 8 canonical debris labels and the extended 50-class OceanGuard taxonomy, where the model occasionally invents intermediate types absent from `DebrisType`; (ii) JSON malformations on the `box_2d` output (missing closing brackets, swapped coordinate order, extra prose around the JSON block) that the parser has to silently repair; and (iii) under-detection on minority classes in our distribution — Metal_Debris (99 instances in our COCO split) and Can (396 instances) are systematically missed at lower thresholds while Bottle and Plastic_Debris dominate. A domain-adapted variant should sharpen the taxonomy alignment, harden the `box_2d` JSON contract, and lift mAP on minority classes without sacrificing the generalist VLM behaviour required for report generation.

Approach

We train a two-stage LoRA adapter over Gemma 4 E2B vision using the Unsloth FastVisionModel stack. The training prompt is byte-identical to the inference prompt declared in `Gemma4VisionDetector.DETECTION_PROMPT`, so adapter behaviour transfers without prompt drift at deployment time.

Stage 1 — geometry and JSON warmup. 10,247 COCO images across the 8 canonical debris classes (CleanSea + Ocean_garbage + Neural_Ocean union), with ground-truth bounding boxes serialized into the exact `box_2d [y_min, x_min, y_max, x_max]` integer 0-1000 schema the inference parser expects. The language model is fine-tuned with LoRA rank 16 over `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj` while the vision tower stays frozen, training 2 epochs at LR 1e-4 with a cosine schedule, AdamW 8-bit, effective batch size 32 via gradient accumulation. The objective is to lock the JSON contract and recover the geometric prior of the dataset.

Stage 2 — granular taxonomy refinement. 449 hand-curated images from `review_dataset/` annotated with material + type at the granular 50-class resolution (e.g., `plastic / bottle_cap`, `metal / fishing_hook`). LoRA rank 8 is added to the vision tower (now unfrozen) on top of the Stage-1 LM adapter; 3 epochs at LR 5e-5, with 10 percent of each batch drawn from a generic VQA mix to mitigate catastrophic forgetting of the open-vocabulary surface that report generation depends on. Training runs on a single RTX 5090 (32 GB), well within the Unsloth memory envelope for Gemma 4 E2B.

Reproducibility

The full pipeline is open and reruns without manual intervention:

- Kaggle-runnable notebook: `docs/submission/notebook_finetune.ipynb` (data prep, both training stages, adapter merge, sanity-check inference)
- Training scripts: `finetune/train_etapa1.py`, `finetune/train_etapa2.py`
- LoRA adapter weights: `huggingface.co/asferrer/gemma-4-E2B-it-oceanguard-marine-debris` (Apache 2.0 code, Gemma Terms weights)
- Dataset prep scripts and split manifests are versioned under `finetune/` in the public repo

Results — Grid de 9 Experimentos (TEST REAL hold-out 1000 imgs)

Métrica oficial sobre TEST hold-out estratificado de 1000 imágenes no vistas durante el entrenamiento. Adapter elegido: **exp07_real100**.

key	mAP@0.5	mAP@0.5:0.95	JSON-validity	n_preds	mean latency (s)
base	0.0924	0.0000	1.000	47	1.96
exp01_synth100	0.1310	0.0000	1.000	80	4.68
exp02_synth100_real10	—	—	—	—	—
exp03_synth100_real25	—	—	—	—	—
exp04_synth100_real50	—	—	—	—	—
exp05_synth100_real75	—	—	—	—	—
exp06_synth100_real100	—	—	—	—	—
exp07_real100	0.1488	0.0000	1.000	90	5.49
exp08_real_synth_minor	—	—	—	—	—
exp09_real_synth_prop	—	—	—	—	—

Per-class mAP@0.5 (todas las filas):

key	Bottle	Can	Fishing_Net	Glove	Mask	Metal_Debris	Plastic_Debris	Tire
base	0.045	0.000	0.091	0.309	0.091	0.000	0.112	0.091
exp01_synth100	0.020	0.000	0.156	0.327	0.091	0.000	0.393	0.061
exp07_real100	0.023	0.000	0.145	0.322	0.091	0.000	0.418	0.191

Δ mAP@0.5 best vs base = **+0.0564** (base=0.0924, best=exp07_real100).

Honest disclosure

El mejor adaptador alcanza Δ = +0.0564 mAP@0.5 respecto a la base, por debajo del umbral interno de +0.20 que nos habíamos fijado. Reportamos la cifra sin maquillaje: dejamos publicado el adaptador junto al grid completo (CSV/MD reproducibles en [finetune/experiments/results/](#)) para que cualquier evaluador pueda confirmar el resultado y comparar configuraciones.

Deployment status

The fine-tuned LoRA adapter is published as a reproducible artifact on HuggingFace and demonstrated end-to-end in a Kaggle notebook. End-to-end conversion to `.litertlm` for on-device LiteRT-LM 0.11.0 deployment requires a `litert-torch` pipeline upgrade that is upstream-in-progress at the Google AI Edge team; the production APK at v0.0.x-beta therefore ships the base Gemma 4 weights. Future work integrates the merged adapter into the on-device runtime once the conversion path is stable, at which point the same `Gemma4VisionDetector` call site picks up the adapted weights with no code change beyond the asset filename.

Unsloth bonus rationale

The training stack uses Unsloth FastVisionModel (not raw `transformers`) for both stages, which satisfies the technical requirement of the Unsloth \$10K bonus track. Beyond the toolchain compliance, Unsloth is the pragmatic choice here: the memory footprint of QLoRA over a vision-language model on a single consumer

GPU is the bottleneck for any independent researcher attempting domain adaptation of an open VLM, and Unsloth removes that bottleneck without sacrificing reproducibility. The domain-adapted variant is therefore both a candidate for the Global Resilience track (a stronger detector on under-represented marine debris classes is a direct resilience contribution) and for the Unsloth bonus track.

10. Reproducibility

Every artifact required to reproduce this submission is open-source under permissive licenses.

Source code. A submission-pinned snapshot of the Android application source is published in the public release repo <https://github.com/asferrer/OceanguardAI> (Apache 2.0), which also hosts the GitHub Releases used for the APK assets and the project landing page. The day-to-day development repository ([asferrer/OceanguardAI-App](#), kept private during the hackathon window) will be opened after the submission deadline. The training and dataset pipeline lives in the same public repo <https://github.com/asferrer/OceanguardAI> (training subtree) under Apache 2.0 and includes the dataset-prep scripts, the erf-free ONNX export, and the COCO conversion utilities.

Notebook. A Kaggle-runnable Jupyter notebook accompanies this submission at [docs/submission/notebook.ipynb](#). It walks through dataset loading, the Gemma 4 `box_2d` parsing for marine-debris detection, and the agentic tool-calling loop in a Python mock, ending with a single-conversation reproduction of the report-generation pipeline (Step 1 tool dispatch + Step 2 writing in the same conversation) over a small synthetic survey.

APK release. A signed reference APK is published as a GitHub Release asset on the public repository (<https://github.com/asferrer/OceanguardAI/releases/latest/download/OceanGuard-AI-latest.apk>) along with the `.litertlm` and GGUF model URLs documented in the README. The release is built automatically by GitHub Actions ([.github/workflows/release.yml](#)) from any pushed `v*` tag; the workflow uses a CI keystore stored in GitHub Secrets and produces a deterministic, signed APK. The build referenced by this submission is the latest beta tag at the time of writing.

Build steps (local). With Java 21 (Temurin or Liberica) and the Android SDK 34+ installed:

```
git clone https://github.com/asferrer/OceanguardAI
cd OceanguardAI/android
./gradlew installDebug
```

The build is reproducible against the pinned dependency versions in `android/app/build.gradle.kts` (Kotlin 2.2.0, Compose BOM 2026.01.01, TFLite 2.17.0, LiteRT-LM 0.11.0). The Gemma 4 E2B `.litertlm` (~2.6 GB) is **downloaded in-app** by `VlmModelManager` from huggingface.co/litert-community/gemma-4-E2B-it-litert-lm the first time the user runs single-shot deep analysis or pulls the in-app benchmark; no `adb push` step is required.

Licenses. Gemma 4 E2B weights are governed by the Gemma Terms of Use; the OceanGuard LoRA adapter (Apache 2.0 training code, Gemma Terms for the merged weights) is published openly. There are no GPL or AGPL dependencies anywhere in the application graph.

References

- [1] Jambeck, J. R., et al. *Plastic waste inputs from land into the ocean*. Science, 347(6223), 768-771. 2015. <https://doi.org/10.1126/science.1260352>
- [2] UN Environment Programme. *From Pollution to Solution: A Global Assessment of Marine Litter and Plastic Pollution*. 2021. <https://www.unep.org/resources/pollution-solution-global-assessment-marine-litter-and-plastic-pollution>
- [3] Kuhn, S., Bravo Rebolledo, E. L., van Franeker, J. A. *Deleterious Effects of Litter on Marine Life*. In Marine Anthropogenic Litter (Springer), 2015. https://doi.org/10.1007/978-3-319-16510-3_4
- [4] Meijer, L. J. J., et al. *More than 1000 rivers account for 80% of global riverine plastic emissions into the ocean*. Science Advances, 7(18). 2021. <https://doi.org/10.1126/sciadv.aaz5803>
- [5] GSMA. *The State of Mobile Internet Connectivity Report*. Annual editions 2021-2024. <https://www.gsma.com/r/somic/>
- [6] Google DeepMind. *Gemma 4 Technical Report*. 2026. <https://ai.google.dev/gemma>
- [7] Ocean Conservancy. *International Coastal Cleanup Annual Reports*. <https://oceanconservancy.org/trash-free-seas/international-coastal-cleanup/>
- [8] Google AI Edge. *LiteRT-LM: On-device LLM runtime*. <https://github.com/google-ai-edge/litert-lm>
- [9] Unsloth AI. *FastVisionModel — efficient VLM fine-tuning*. <https://github.com/unslothai/unsloth>